Software Requirements Engineering

What you know,



System Development Life Cycle



Software development process models

What are we going to learn today?

After this lesson you should be able to;



Define

What a software requirement is, and why it matters before coding begins.



Identify

Functional Requirements



Identify

Non-Functional Requirements

Think



Imagine you're describing Uber to someone who has never used the app.



What can users do using Uber?



How does it behave?

Let’s order a pizza

Suppose you're ordering a pizza online.
What would you specify?

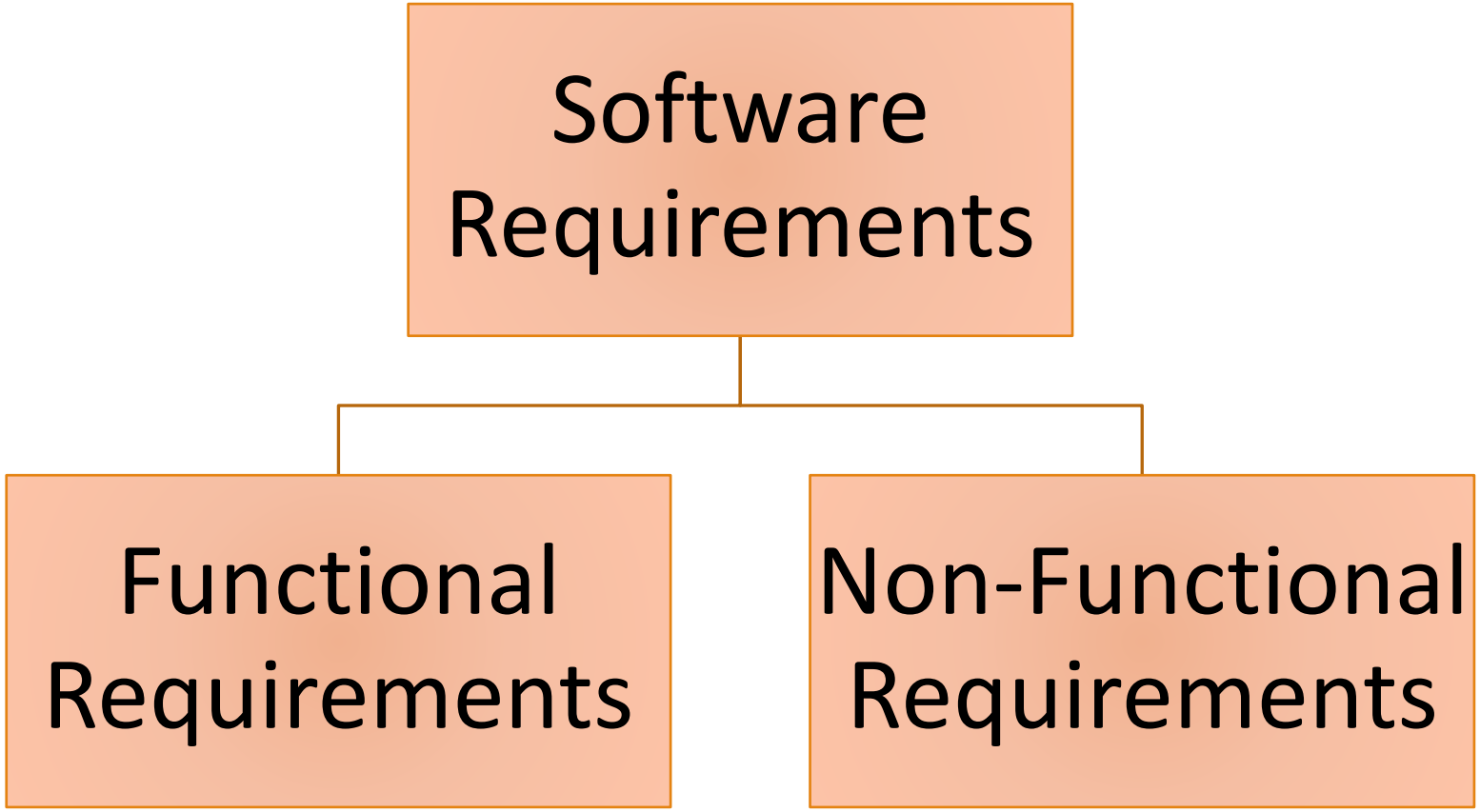


What is a Software Requirement?

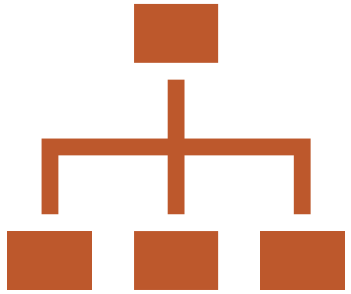
A software requirement describes what the system should do and how well it should perform or the constraints or qualities it should satisfy.



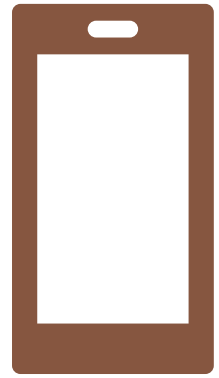
Software Requirements



Functional Requirements



Describe **what the system does**



Product Features

Examples



Logging in



Add products



Search products



Generate reports



Make payments

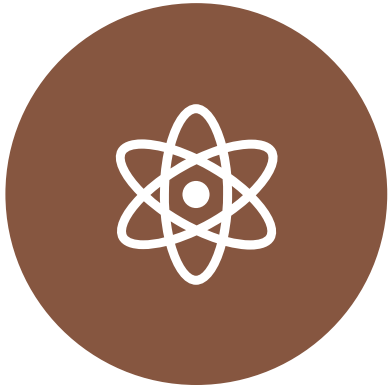
Pair Activity

Write five functional requirements of an online Banking Application.

Non-Functional Requirements



Describe **how well the system performs**



System properties



User expectations



Non-Functional Requirements

Non-Functional Requirements specify the **quality attributes or constraints** that define **how well** a software system should perform.

Non-Functional Requirements

Performance

The system shall display search results within 2 seconds for 95% of requests.

Security

The system shall lock an account after 5 unsuccessful login attempts.

Usability

90% of first-time users shall complete registration in under 3 minutes without assistance

Reliability

The system shall recover from failures within 30 seconds.

Scalability

The system shall support 10,000 concurrent users without exceeding a 3-second response time

Portability

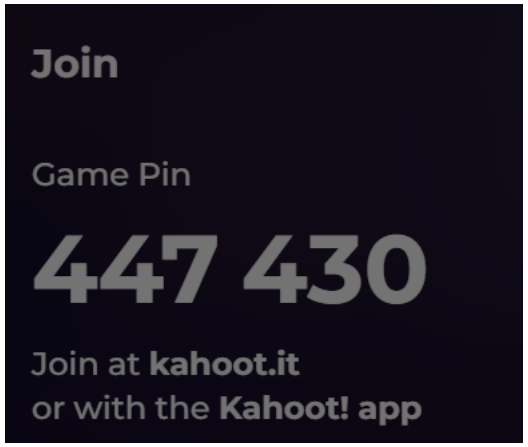
The application shall run on Chrome, Edge, Firefox, and Safari without functional differences.

Class Discussion

Let’s classify the following requirements as Functional and Non-Functional.

- Customers can add items to the shopping cart.
- The application should be available 24/7.
- Users can update their profile.
- The interface should be easy to use

Individual Activity



Summary

	Functional (FR)	Non-Functional (NFR)
Answers	What does it do?	How well does it do it?
Describes	Features & behaviours	Qualities & constraints
Tested by	Checking a feature works	Measuring performance/quality
Example	*"User can reset password"*	*"Reset email arrives in <5 sec"*

Key Takeaway

Functional = **WHAT** the system does. **Non-Functional** = **HOW** WELL the system does it.

Every requirement you write should clearly answer one of those two questions.

MoSCoW Method

- A prioritization technique to determine which requirements or features are most important to deliver in a project.
- It helps teams and stakeholders agree on what is truly essential for success and what can be deferred or dropped if necessary.

Software Requirements

Requirements are categorized logically as

Must Have :Software cannot be said operational without them.	**Should have** : Enhancing the functionality of software.	**Could have** :Software can still properly function with these requirements.	**Wish list** : These requirements do not map to anyobjectives of software.

While developing software, ‘Must have’ must be implemented, ‘Should have’ is a matter of debate with stakeholders and negation, whereas ‘could have’ and ‘wish list’ can be kept for software updates.

MosCoW Method

Must have:	These are *critical requirements* that the system **must** include for it to be successful. Without these, the product is not viable or deliverable.
Should have:	These are *important* but not vital features. The system can still work without them, but their absence may reduce usability or efficiency.
Could have:	These are *desirable but optional* features. They can improve user satisfaction or add convenience if time and resources permit.
Wish list (Won't have this time)	These are *lowest priority* or deferred features. They are not planned for the current release but might be considered in the future.

What is Requirements Engineering ?



The process of defining, documenting, and maintaining requirements in the engineering design process.



The process of establishing the services that a customer requires from a system and the constraints under which it operates and is developed.

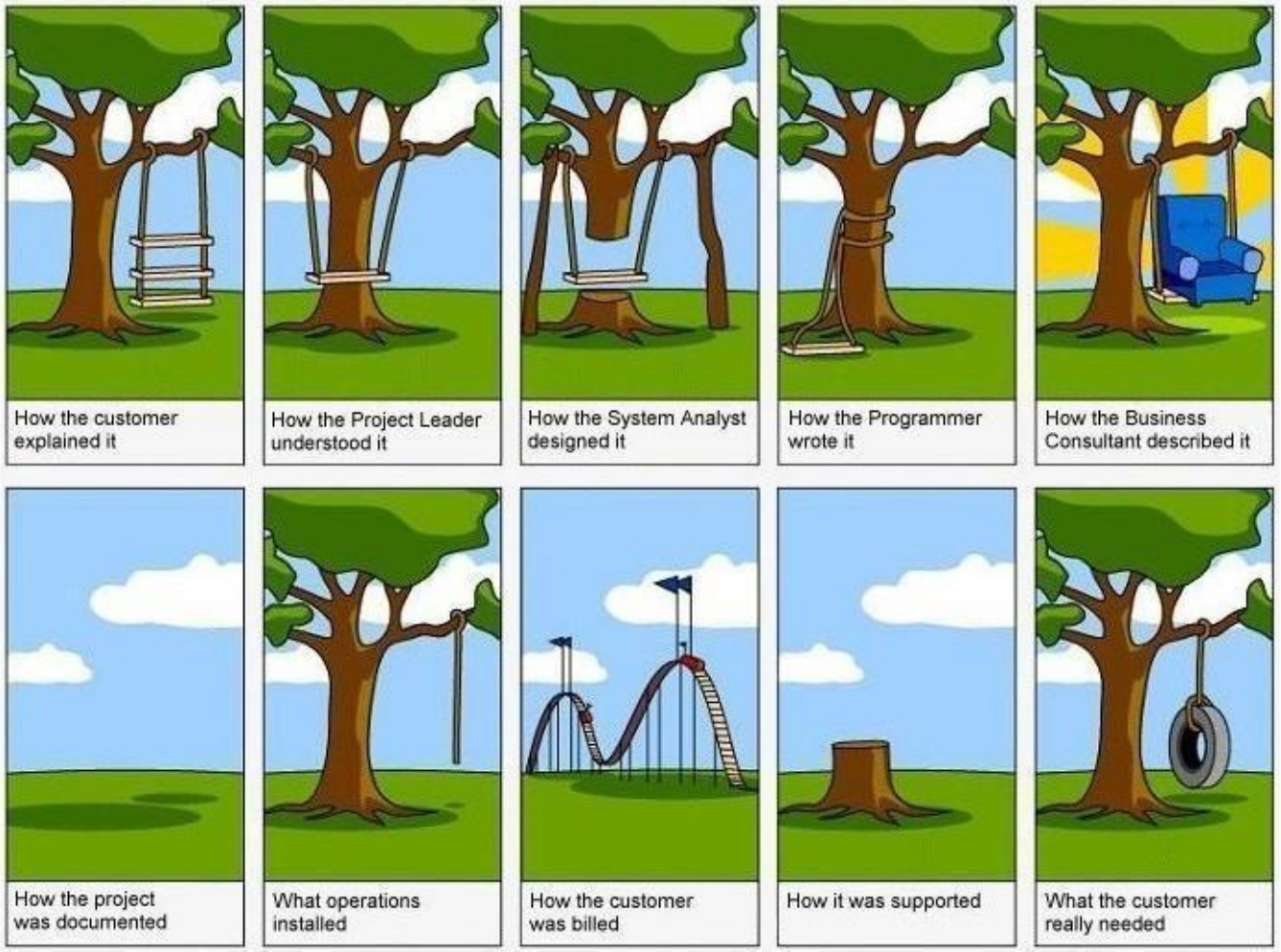


Processes used to discover, analyse and validate system requirements.

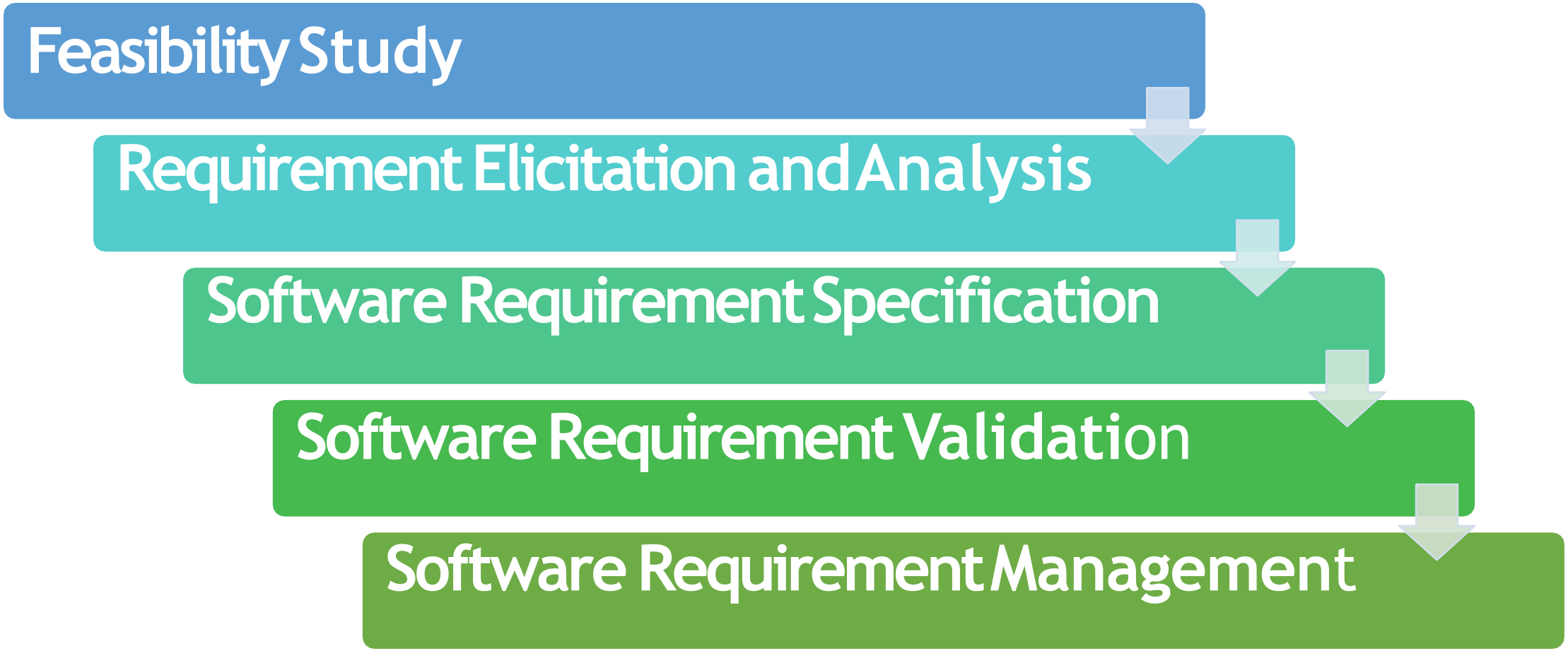


Here the requirements of the software systems are gathered from the client, analyzed and documented.

Why Requirements Engineering is important?



Requirements Engineering Process



1. Feasibility Study

An estimate is made whether the identified requirements can be achieved using the technologies, under the specified budget etc.

The feasibility study should be cheap and quick.

Feasibility study decides whether the proposed system is worthwhile or not.

It's a short, focused study that checks

- If the system contributes to organisational objectives
- If the system can be engineered using current technology and within budget
- If the system can be integrated with other systems that are used

Helps in deciding whether to go ahead or with the project.

Types of Feasibility



Technical Feasibility - Evaluates the current technologies, whichare needed to accomplish customer requirements within the time and budget.



Operational Feasibility - Assesses the range in which the required software performs a series of levels to solve business problems and customer requirements.



Economic Feasibility - Decides whether the necessarysoftware can generate financial profits for anorganization.

2. Requirements Elicitation and Analysis

- This is also known as the **gathering of requirements**.
- Here, requirements are identified with the help of customers and existing systems processes, if available.
- Analysis ofrequirements starts with requirement elicitation.
- The requirements are analyzed to identify inconsistencies, defects, omission, etc.

Problems in Elicitation and Analysis



Getting all, and only, the right people involved.



Stakeholders often don't know what they want.



Stakeholders express requirements in their terms. Stakeholders may have conflicting requirements.



Requirement change during the analysis process.



Organizational and political factors may influence system requirements.

Requirement Elicitation Techniques



Interviews



Surveys



Questionnaires



Task analysis



Domain Analysis



Brainstorming



Prototyping



Observations

3. Software Requirement Specification

Software requirement specification is a document which is created by a software analyst after the requirements collected from the various sources.

SRS defines how the intended software will interact with external interfaces, speed of operation, response time of system, portability of software across various platforms, maintainability, speed of recovery after crashing, Security, Quality, Limitations etc.

The requirement received by the customer is in natural language.

It is the duty of the analyst to write the requirement in technical language so that they can be understood and used by the development team.

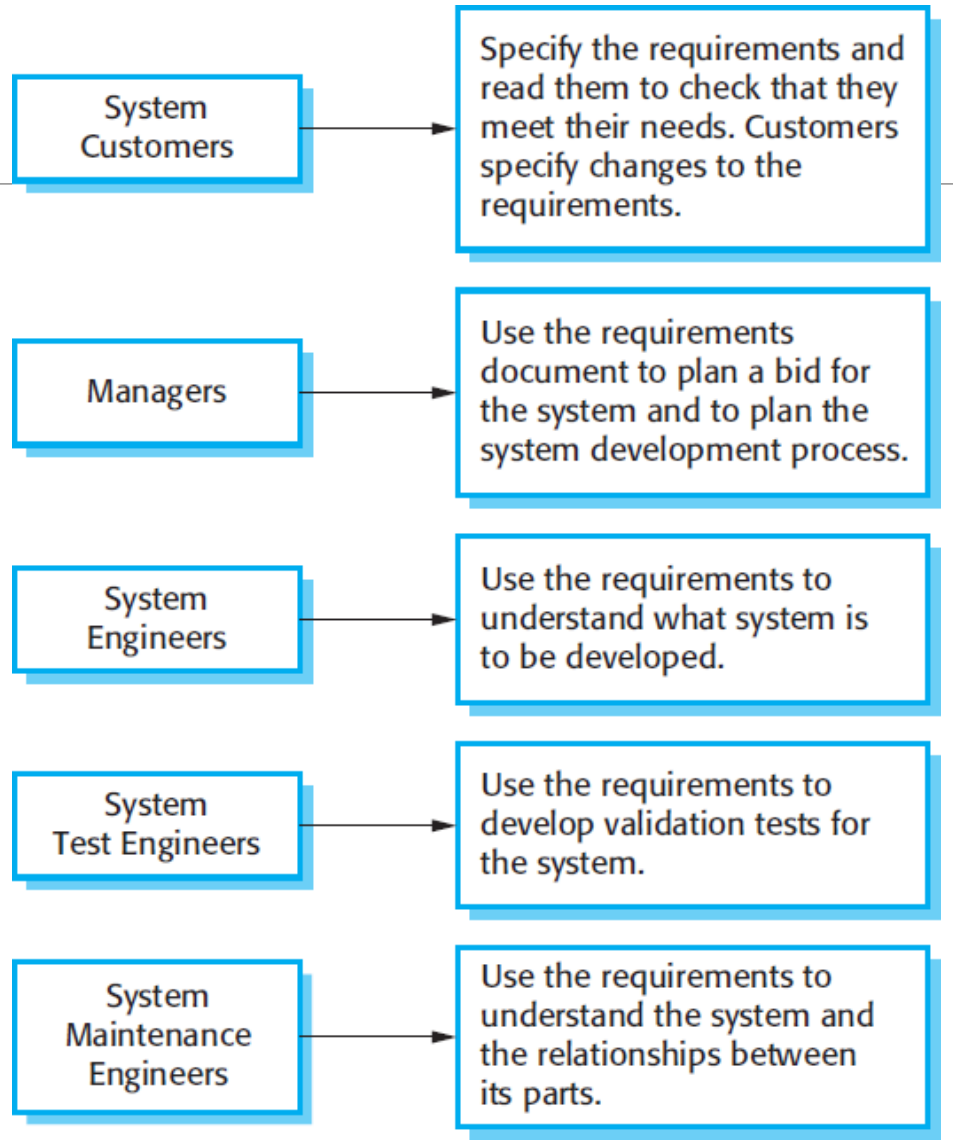
Characteristics of a good SRS

- User requirements are expressed in natural language.
- Technical requirements are expressed in structured language, which is used inside the organization.
- Design description should be written in Pseudo code.
- Format of Forms and GUI screen prints.
- Conditional and mathematical notations for DFDs etc.

Characteristics of a SRS

- Clear
- Correct
- Consistent
- Coherent
- Comprehensible
- Modifiable
- Verifiable
- Prioritized
- Unambiguous
- Traceable
- Credible source

Users of SRS


System Customers
Specify the requirements and read them to check that they meet their needs. Customers specify changes to the requirements.
Managers
Use the requirements document to plan a bid for the system and to plan the system development process.
System Engineers
Use the requirements to understand what system is to be developed.
System Test Engineers
Use the requirements to develop validation tests for the system.
System Maintenance Engineers
Use the requirements to understand the system and the relationships between its parts.

Developing Requirements

Requirements can be modelled as,

- Natural Language Requirements
- Modelled Requirements

Modelled Requirements

Some models used at this stage include

- UML diagrams
- ER diagrams
- Data Flow Diagrams (DFDs)
- Function Decomposition Diagrams (FDDs)
- Data Dictionaries etc.
- Graphs
- Formulas
- Code/Pseudocode

IEEE SRS template can be used to create an SRS.

Activity

Index	Requirement
R1	The system shall allow students to log in using their student ID and password.
R2	The system shall be very fast.
R3	The system shall predict exam questions with 100% accuracy.
R4	The system shall allow students to register for courses.
R5	The system shall never fail.
R6	The system shall send a confirmation email after successful course registration.

4. Software Requirement Validation

- After the SRS is designed, the requirements in the SRS are validated because the users may have demanded illegal, impossible solution or experts may misinterpret the needs.
- Requirements can be the check against the following conditions.
  - If they can practically implement
  - If they are correct and as per the functionality and specially of software
  - If there are any ambiguities
  - If they are full
  - If they are descriptive

4. Software Requirement Validation

- Concerned with demonstrating that the requirements define the system that the customer really wants
- Requirements error costs are high so validation is very important
- Fixing a requirements error after delivery may cost up to 100 times the cost of fixing an implementation error.

Requirements validation techniques



Requirements reviews

Systematic manual analysis of the requirements



Prototyping

Using an executable model of the system to check requirements.



Test-case generation

Developing tests for requirements to check testability



Automated consistency analysis

Checking the consistency of a structured requirements description

Requirements reviews

Regular reviews should be held while the requirements definition is being formulated

Both client and contractor staff should be involved in reviews

Reviews may be formal (with completed documents) or informal.

Good communications between developers, customers and users can resolve problems at an early stage

Review checks



Verifiability. Is the requirement realistically testable?



Comprehensibility. Is the requirement properly understood?



Traceability. Is the origin of the requirement clearly stated?



Adaptability. Can the requirement be changed without a large impact on other requirements?

Review Checks



Validity

Does the system provide the functions which best support the customer's needs?



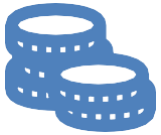
Consistency

Are there any requirements conflicts?



Completeness

Are all functions required by the customer included?



Realism

Can the requirements be implemented given available budget and technology

Traceability



Traceability is concerned with the relationships between requirements, their sources and the system design



Source traceability

Links from requirements to stakeholders who proposed these requirements



Requirements traceability

Links between dependent requirements



Design traceability

Links from the requirements to the design

5. Software Requirement Management



Requirement management is the process of analyzing, documenting, tracking, prioritizing and agreeing on the requirements and communication to relevant stakeholder.



New requirements emerge during the process as business needs a change, and a better understanding of the system is developed.

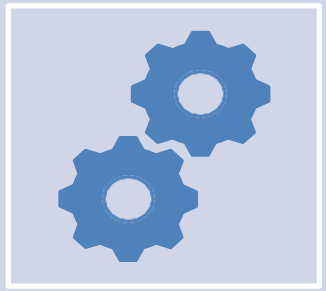


The priority of requirements from different viewpoints changes during development process.



The business and technical environment of the system changes during the development.

Requirements Management



Requirements management is the process of managing changing requirements during the requirements engineering process and system development



Requirements are inevitably incomplete and inconsistent

New requirements emerge during the process as business needs change and a better understanding of the system is developed.

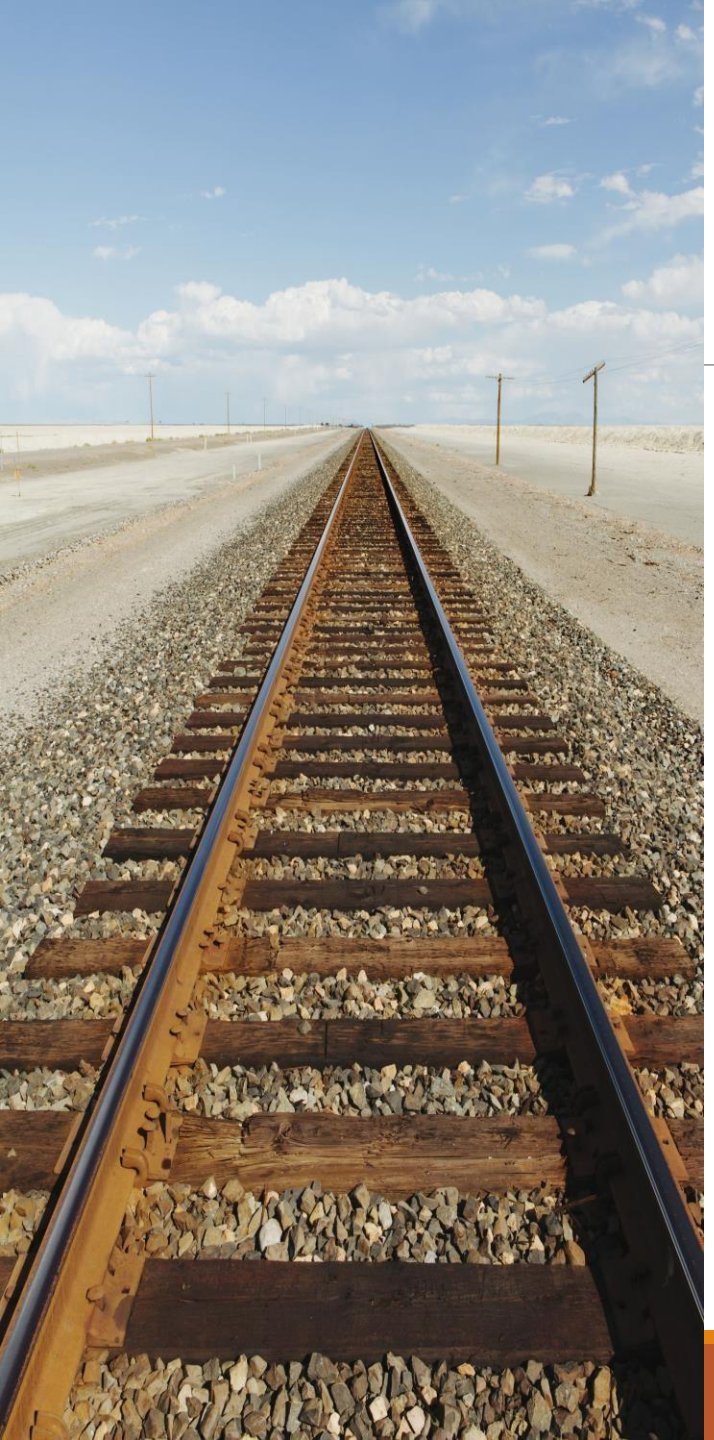
Different viewpoints have different requirements and these are often contradictory

Requirements change

The priority of requirements from different viewpoints changes during the development process

System customers may specify requirements from a business perspective that conflict with end-user requirements

The business and technical environment of the system changes during its development



Requirements management planning

During the requirements engineering process, you have to plan:

- Requirements identification
  - How requirements are individually identified
- A change management process
  - The process followed when analysing a requirements change
- Traceability policies
  - The amount of information about requirements relationships that is maintained
- CASE tool support
  - The tool support required to help manage requirements change



Questions ?



Thank you